

React.js

Day 1 — Introduction to React

Student Notes · Beginner Friendly · Interview Ready

What is React | Library vs Framework | SPA | Virtual DOM | Components | JSX | Babel | Import & Export

What You Will Learn

- What is React?
- Why React is needed?
- Library vs Framework
- Traditional Websites vs React Applications
- Virtual DOM
- React Rendering Flow
- Creating a React Project using Vite
- React Folder Structure
- Components & Functional Components
- React Fragment
- JSX & Babel
- Import & Export

1. What is React?

Open-source UI library by Meta

React is an open-source JavaScript library developed by Meta for building fast, reusable User Interfaces (UI).

React is mainly used to:

- Build modern websites
- Create reusable UI components
- Develop Single Page Applications (SPA)
- Make applications faster

2. Why React is Needed?

The problem with traditional websites

The Problem — Traditional Websites

- Every page (Home, About, Login) is a separate HTML file
- Moving between pages reloads the entire browser window
- This causes slower performance, more loading time, poor user experience

Traditional Website Flow

Home.html → About.html → Login.html

Every navigation = Full page reload

The Solution — React

- Only one HTML page exists (index.html)
- React replaces only the required section without reloading
- This concept is called a Single Page Application (SPA)

React Application Structure

```
index.html
├── Navbar    ← component
├── Banner    ← component
├── Blog      ← component
├── About     ← component
├── Contact   ← component
└── Footer    ← component
```

3. Library vs Framework

Understanding where React fits

Library (React)	Framework (Angular)
You decide when and how to use it	Framework decides the structure for you
You control the application flow	Framework controls your code
More flexible	More opinionated / strict
Smaller — only UI layer	Complete solution — routing, HTTP, UI

Easy Formula:

Library → You control the flow.

Framework → Framework controls your code.

4. Components

The building blocks of every React app

A Component is a small, reusable piece of UI. Think of components like LEGO blocks — each piece is independent and can be assembled to build the full page.

Example — Website as Components

```
Website
Navbar    → reusable in all pages
Banner    → hero section
About     → about section
Blog      → blog section
Contact   → contact section
Footer    → reusable in all pages
```

Advantages of Components

- Reusable — write once, use anywhere
- Independent — changes in one don't break others
- Easy to manage and test
- Easy to update

Types of Components

Functional Component ✓	Class Component ✗
Uses JavaScript functions	Uses JavaScript classes
Modern & recommended approach	Old approach — rarely used in new projects
Simpler & cleaner syntax	More verbose
Use this in all your projects	Only relevant for interview questions

5. Functional Components & Rules

Writing your first React component

Two Key Rules

- Component name must start with a Capital Letter
- Must return only one parent element

Example

```
function Welcome() {  
  return (  
    <div>  
      <h1>Hello</h1>  
      <p>Welcome to React</p>  
    </div>  
  );  
}
```

React Fragment — When You Don't Want an Extra div

Sometimes adding an extra `<div>` wrapper is unnecessary. Use a React Fragment instead.

```
<>  
  <h1>Hello</h1>  
  <p>Welcome</p>  
</>
```

Note: *Fragment groups elements without creating an extra HTML element in the page.*

6. Virtual DOM

React's secret to fast performance

Virtual DOM is a lightweight JavaScript copy of the Real DOM. Instead of updating the browser directly, React first makes changes in the Virtual DOM, then applies only the changed parts to the Real DOM.

Why is this faster?

- Real DOM updates are slow — the entire page can re-render
- Virtual DOM is just a JavaScript object — very fast to update
- React identifies exactly what changed and applies only that

React Rendering Flow

```
Real DOM
  ↓ copy/replica
Virtual DOM
  ↓ state or props change
Updated Virtual DOM
  ↓ Diffing Algorithm compares old vs new
Reconciliation
  ↓ apply only the changed parts
Updated Real DOM
```

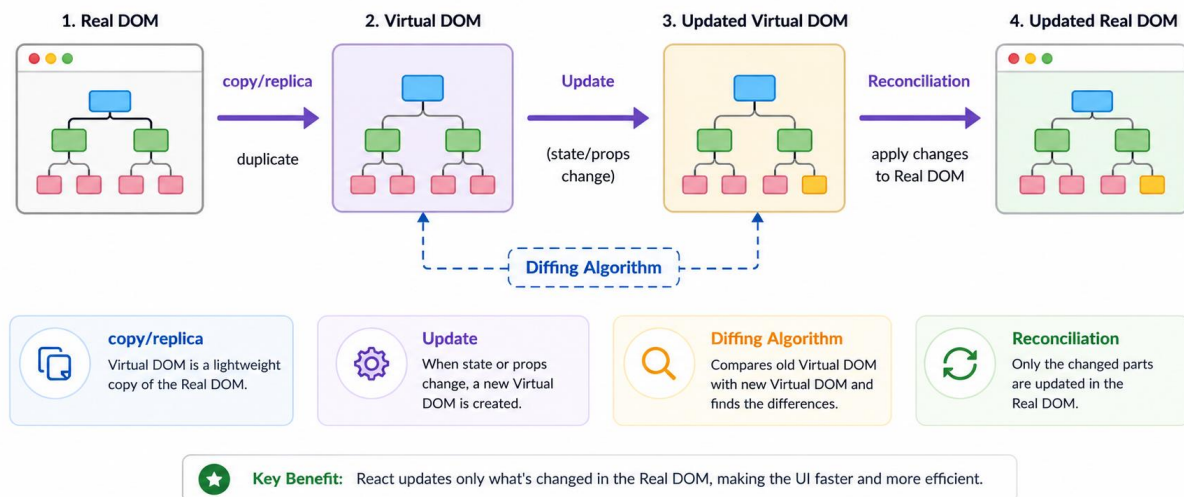
Diffing Algorithm

- Compares old Virtual DOM with new Virtual DOM
- Finds exactly what has changed
- Does not touch unchanged parts

Reconciliation

- After diffing identifies changes, reconciliation applies them
- Only the changed elements are updated in the Real DOM
- This is what makes React fast and efficient

React Rendering Flow (Virtual DOM Process)



React Rendering Flow — Virtual DOM Process

7. Creating a React Project with Vite

The fastest way to start a React app

Note: We use Vite (not Create React App) because Vite is faster, lighter, and the current industry standard.

Steps

Step 1 — Create the project

```
npm create vite@latest
```

Step 2 — Choose options

```
Framework: React
Variant:   JavaScript
```

Step 3 — Install packages

```
npm install
```

Step 4 — Run the project

```
npm run dev
```

8. React Folder Structure

Understanding what each file does

File / Folder	Purpose
node_modules/	All installed packages and libraries (never edit this)
public/	Static files — images, videos, favicons
src/	Heart of the app — where you write all your code
src/assets/	Images, videos, icons used inside components
src/main.jsx	Entry point — connects React with index.html
src/App.jsx	Root component — your app starts here
index.html	Single HTML file — contains only <code><div id="root"></div></code>
package.json	Project name, version, dependencies, scripts
.gitignore	Files Git should ignore (e.g. node_modules)

How it all connects

```
index.html → has <div id="root"></div>
main.jsx   → injects React app into that root div
App.jsx     → root component – everything starts here
```

9. JSX

Writing HTML inside JavaScript

JSX stands for JavaScript XML. It allows us to write HTML-like syntax directly inside JavaScript files.

Example

```
// JSX – looks like HTML but lives inside JavaScript
function App() {
  return <h1>Hello React</h1>;
}
```

JSX vs HTML — Key Differences

HTML	JSX
class=""	className="" (class is a reserved word in JS)
for=""	htmlFor="" (for is a reserved word in JS)
Tags not required to close	All tags must be self-closed e.g.
No expressions	Can embed JS expressions: {2 + 2}

10. Babel

The JSX-to-JavaScript converter

Babel is a JavaScript compiler. Browsers do not understand JSX directly. Babel automatically converts JSX into plain JavaScript that all browsers can run.

How it works

```
JSX (what you write)
  ↓ Babel compiles
JavaScript (what the browser reads)
```

Example

Before Babel (JSX)

```
<h1>Hello React</h1>
```

After Babel (JavaScript)

```
React.createElement("h1", null, "Hello React")
```

Note: Vite handles Babel automatically. You don't need to configure it yourself.

11. Import & Export

Sharing code between files

React projects are split into multiple files. To use code from another file, we use Export and Import.

Default Export

- Only one default export allowed per file
- Imported without curly braces { }
- You can rename it on import

Exporting

```
export default Header;
```

Importing

```
import Header from "./Header";
```

Named Export

- Multiple named exports allowed per file
- Imported with curly braces { }
- Name must match exactly

Exporting

```
export const Add = () => {};  
export const Sub = () => {};
```

Importing

```
import { Add, Sub } from "./Math";
```

Default Export vs Named Export

Default Export	Named Export
One per file	Multiple per file
No curly braces on import	Curly braces required on import
Name can be changed on import	Name must match exactly
<code>export default Comp</code>	<code>export const Comp = ...</code>

Common Beginner Mistakes

- ✗ Writing component names in lowercase (header instead of Header)
- ✗ Forgetting to export a component
- ✗ Mixing default and named imports
- ✗ Returning multiple parent elements without a Fragment or wrapper
- ✗ Editing files inside `node_modules`

Interview Questions & Answers

Q: What is React?

A: React is an open-source JavaScript library developed by Meta for building fast and reusable user interfaces.

Q: Why is React called a Library?

A: Because the developer controls when and how to use React. React does not control the entire application structure.

Q: What is a Single Page Application (SPA)?

A: An application that loads only one HTML page and updates content dynamically without reloading the entire page.

Q: What is a Component?

A: A reusable building block of a React application. Each component represents a small piece of the UI.

Q: What is Virtual DOM?

A: A lightweight JavaScript copy of the Real DOM. React updates the Virtual DOM first, then applies only the changed parts to the Real DOM.

Q: What is the Diffing Algorithm?

A: It compares the old and new Virtual DOM and identifies exactly what has changed.

Q: What is Reconciliation?

A: The process of applying only the changed parts of the Virtual DOM to the Real DOM.

Q: What is JSX?

A: JSX (JavaScript XML) is a syntax that lets us write HTML-like code inside JavaScript files.

Q: What is Babel?

A: Babel is a JavaScript compiler that converts JSX into plain JavaScript that browsers can understand.

Q: Difference between Default Export and Named Export?

A: Default Export: one per file, imported without curly braces, name can change.
Named Export: multiple per file, imported with curly braces, name must match.

Q: What is the difference between a Library and a Framework?

A: Library: you control the flow (React). Framework: the framework controls your code (Angular).

Practice Questions

1. What problem does React solve compared to traditional websites?
2. Explain the difference between a Library and a Framework.
3. What is a Component? Give a real-life example.
4. What is a Single Page Application?
5. Explain the React Rendering Flow step by step.
6. Why is Virtual DOM faster than updating the Real DOM directly?

7. What is the purpose of main.jsx?
 8. What is the purpose of App.jsx?
 9. What is the difference between JSX and HTML?
 10. Write one example each for a Default Export and a Named Export.
-

Day 1 Summary

- ✓ React is a JavaScript library for building user interfaces — developed by Meta.
- ✓ React uses reusable components as building blocks.
- ✓ SPA — only one HTML file, React swaps components without reloading.
- ✓ Virtual DOM makes React fast — updates only what changed.
- ✓ Diffing Algorithm finds what changed. Reconciliation applies it.
- ✓ Vite is used to create React projects — fast and lightweight.
- ✓ main.jsx is the entry point. App.jsx is the root component.
- ✓ JSX lets us write HTML-like syntax inside JavaScript.
- ✓ Babel converts JSX into JavaScript the browser can run.
- ✓ Default Export — one per file, no curly braces.
- ✓ Named Export — multiple per file, curly braces required.